

Laboratório de Programação Concorrente

Lab5

Sum of a File -

26.1

Objetivo

Neste laboratório, o objetivo é que vocês se familiarizem com a sintaxe e o estilo de diferentes linguagens de programação ao desenvolver soluções concorrentes utilizando múltiplas threads. Para isso, disponibilizamos uma implementação do **cálculo de sum de arquivos** em três linguagens diferentes: **Python, Java e C**. Para cada linguagem, a partir dessas versões sequenciais, vocês devem desenvolver uma versão concorrente que explore o uso de múltiplas threads para melhorar o desempenho. O sum de um arquivo é uma representação numérica do seu conteúdo. Existem diversas formas de calculá-lo, mas neste laboratório adotamos uma abordagem simples: a soma de todos os bytes do arquivo.

A implementação fornecida atualmente realiza o cálculo de forma sequencial, processando os arquivos um por vez. **Sua tarefa é adaptar essa solução para que o cálculo dos sums dos arquivos seja feito de forma concorrente**, utilizando threads, de modo a explorar paralelismo e reduzir o tempo total de execução.

Visão geral do código base

No código base vocês encontrarão implementações seriais em Python, Java e C. Cada implementação recebe como argumentos os "path" de arquivos que devem ter o sum calculado. A sua implementação deve adicionar concorrência a esse processo.

A entrega, detalhada nas seções seguintes, envolverá o código fonte. Iremos avaliar tanto as possibilidades de plágio entre os alunos quanto a geração automática de código.

<https://classroom.github.com/a/JptzqiYa>

O código está organizado na seguinte hierarquia:

- ▼ Lab5
 - ▼ src
 - ▼ c
 - ▼ concurrent
 - build.sh
 - run.sh
 - sum.c
 - ▼ serial
 - build.sh
 - run.sh
 - sum.c
 - ▼ java
 - ▼ concurrent
 - Sum.java
 - build.sh
 - run.sh
 - ▼ serial
 - Sum.java
 - build.sh
 - run.sh
 - ▼ python
 - ▼ concurrent
 - buid.sh
 - run.sh
 - sum.py
 - ▼ serial
 - buid.sh
 - run.sh
 - sum.py
 - > support-doc
 - make_dataset.sh
 - run_all.sh
 - README.md

- `src/*/serial/`:
São os diretórios com as implementações serial em cada linguagem, esse é o ponto de partida para entender o funcionamento do cálculo do sum dos arquivos.
- `src/*/concurrent/`:
São os diretórios onde vocês implementarão as soluções concorrentes. **Note que, inicialmente, esse diretório contém a mesma implementação que a serial, então vocês devem alterá-la para implementar a concorrência!**
- `make_dataset.sh`:
Script para gerar um conjunto de n arquivos que podem ser considerados para input das execuções. Os arquivos serão gerados em um diretório chamado dataset. Para executar esse script, informe o número de arquivos a serem gerados como argumento, por exemplo:

```
bash make_dataset.sh 10
```
- `run_all.sh`:
Script para compilar e executar as implementações serial e concorrente em todas as linguagens de programação, em seus respectivos diretórios. Este script considera que os arquivos do diretório dataset serão passados como argumentos das execuções.
- `support-doc/`:
Diretório onde você encontra documentação para lidar com threads na linguagem de programação do código base.

Preparação

1. Clone o repositório do código base

```
git clone [link do repositório]
```

Faça uma comparação de desempenho entre as versões serial e concorrente (detalhes abaixo):

2. Execute o script `make_dataset.sh` para gerar arquivos

```
bash make_dataset.sh 10
```

3. Execute o script `run_all.sh` para executar todos os cenários considerando os arquivos `dataset` como argumentos das execuções

```
bash run_all.sh
```

Comparação de Desempenho

Ao executar o script `run-all.sh`, vocês devem focar na comparação do tempo real (é o tempo que você veria em um cronômetro), podendo desconsiderar neste momento os tempos de `user` (o tempo total que o processo gastou utilizando a CPU em modo usuário) e `sys` (o tempo total que o processo gastou utilizando recursos do kernel).

Prazo

19/05/2026 às 16h00

Entrega

A entrega se dará através das atualizações do código no próprio repositório da dupla no Github Classroom (<https://classroom.github.com/a/JptzqiYa>).